

shogiAI 資料庫系統報告

遠端棋譜蒐集、MySQL 資料庫、API 與棋譜管理系統

shogiAI 專案

2026 年 6 月

摘要

本報告說明 shogiAI 專案的資料庫系統。專案以 CSA 將棋棋譜為核心資料，透過遠端大數據主機從 shogidb2 蒐集棋局，將棋譜匯入 MySQL，再由 Python HTTP API 提供前端棋譜瀏覽、查詢、CSA 上傳、自我對弈、AI 對弈、模型互打與訓練資料匯出等功能。資料庫設計採用關聯式模型，將一盤棋拆成使用者、棋手、對局、走法與局面五種資料，使系統能同時滿足資料保存、互動回放、資料品質控管、AI 訓練與開局庫統計需求。

目录

1	專案目標	3
2	整體系統架構	3
3	遠端資料蒐集流程	3
4	本地與遠端同步機制	4
5	資料庫設計方法	4
5.1	走法與局面的核心關係	5
6	資料表設計	5
6.1	users	5
6.2	players	6
6.3	game_records	6
6.4	moves	6
6.5	positions	7
7	CSA 匯入 MySQL 流程	7

目录	2
8 資料品質與去重	8
8.1 合法手驗證	8
8.2 檔名去重	8
8.3 內容去重	8
9 自動匯入與監控	8
10 後端 API 設計	9
11 前端棋譜瀏覽器	9
12 資料庫如何支援 AI	10
12.1 訓練資料匯出	10
12.2 Opening Book	10
12.3 AI 對弈資料服務	10
13 安全性與維護	10
13.1 SQL 注入防護	10
13.2 密碼管理	11
13.3 上傳安全	11
14 效能與索引建議	11
15 限制與未來改進	12
16 結論	12
A 常用操作指令	12
A.1 匯入 CSA 到 MySQL	12
A.2 啟動瀏覽器 API	12
A.3 匯出 policy 訓練資料	13

1 專案目標

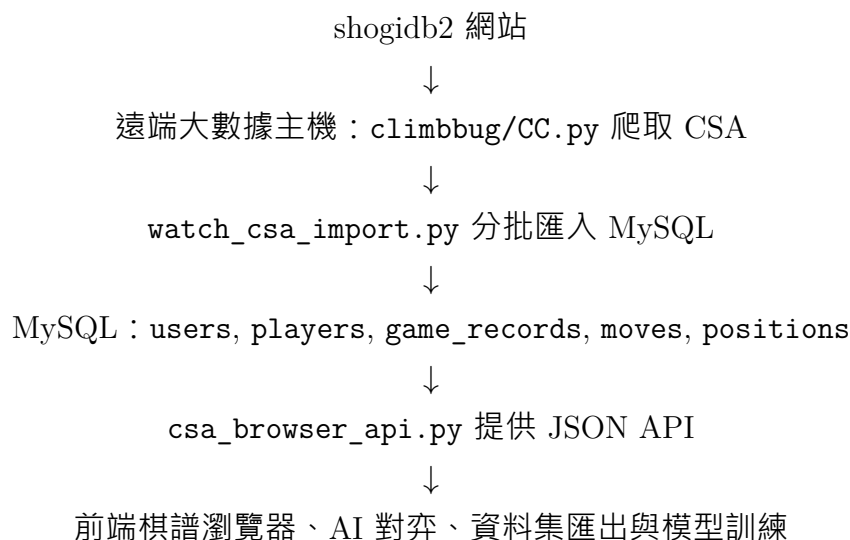
shogiAI 的資料庫系統不是單純保存棋譜檔案，而是把原始 CSA 棋譜整理成可查詢、可回放、可訓練、可驗證的資料資產。整體目標分成四個層次：

1. **資料蒐集**：從 shogidb2 網站取得棋局資料，轉成 CSA 檔案並保存。
2. **資料管理**：將 CSA 檔案匯入 MySQL，以正規化資料表保存棋手、對局、走法與局面。
3. **資料服務**：透過 HTTP JSON API 讓前端查詢棋局、回放任意手數、上傳新棋譜與取得資料庫統計。
4. **AI 支援**：由資料庫匯出 policy 訓練資料集，並從資料庫統計 opening book 供 AI 對弈使用。

資料庫因此是整個專案的中樞。爬蟲、前端、後端、訓練程式與 AI 搜尋系統都圍繞同一份 MySQL 資料運作。

2 整體系統架構

專案由本地開發環境、遠端大數據主機、MySQL 主機與 GitHub 四個部分組成。



本地與遠端的程式碼透過 GitHub main 分支同步。遠端負責長時間蒐集資料與訓練，本地負責開發、展示、撰寫報告與使用最新模型。

3 遠端資料蒐集流程

遠端爬蟲位於 climbbug/CC.py。它的目的，是從 shogidb2 網站取得公開棋局並轉成 CSA 格式。流程如下：

1. 使用 `requests` 下載 shogidb2 的最新棋譜列表頁。
2. 使用 `BeautifulSoup` 解析 HTML 中的棋局連結。
3. 將棋局 URL 寫入 `game_urls.txt`，避免重複掃描。
4. 使用 `Selenium` 啟動 headless Chrome 開啟棋局頁面。
5. 從 Chrome performance log 中讀取 WebSocket frame。
6. 在 frame payload 中尋找包含 moves 與 csa 的棋局資料。
7. 將棋手、賽事、地點、時間、戰型與每一步 CSA 走法組合成完整 CSA 文字。
8. 寫入 `climbbug/data/gameN.csa`。
9. 若抓取失敗，將 URL 寫入 `failed_urls.txt`。

這個設計同時使用靜態解析與瀏覽器自動化。列表頁可用 `requests` 和 HTML parser 快速取得；棋局內容則需要 `Selenium`，因為實際資料藏在瀏覽器接收到的動態訊息中。

4 本地與遠端同步機制

專案使用 GitHub 作為共同來源。同步腳本包含：

- `sync_repo.ps1`：本地 Windows 使用，負責 `git add`、`commit`、`pull --rebase`、`push`。
- `sync_repo.sh`：Linux 環境使用，功能與 PowerShell 版本相同。
- `sync_remote.sh`：遠端大數據主機使用，會先把 live crawler 目錄 `$HOME/climbbug` 的資料複製進 repo 的 `climbbug/`，再提交推送，最後把 repo 內容同步回 live crawler 目錄。

這樣做的好處是遠端與本地不直接互相覆蓋，而是透過 Git 歷史保留每次資料與程式變更。如果兩邊改到同一檔案，Git rebase 會停下來要求解衝突，避免資料被靜默覆寫。

5 資料庫設計方法

本專案採用關聯式資料庫 MySQL。設計時先拆出 CSA 棋譜中的主要實體：

- 使用者或匯入者。
- 棋手。

- 一盤對局的 metadata。
- 每一步走法。
- 每一手後的局面。

因此資料庫拆成五張主要資料表：users、players、game_records、moves、positions。這樣符合正規化精神：棋手名稱不重複存在每一步資料中，對局資料不混入局面資料，走法與局面也分開保存。

5.1 走法與局面的核心關係

本專案最重要的資料模型是：

$Position_t + Move_{t+1} \rightarrow Position_{t+1}$

positions 的第 0 筆代表初始局面，第 t 筆代表走完第 t 手後的局面；moves 的第 $t + 1$ 筆代表從 Position_t 出發的下一步。這個設計使系統可以：

- 直接查詢任意手數，不必每次從第一手重播。
- 從 positions join moves 匯出「局面、下一手」訓練資料。
- 用 SFEN hash 統計重複局面或建立 opening book。

6 資料表設計

6.1 users

users 保存系統使用者或匯入者，目前主要用於記錄資料來源與上傳者。

欄位	型別	用途
user_id	INT PK	使用者主鍵，自動遞增。
username	VARCHAR	使用者或匯入程式名稱。
email	VARCHAR UNIQUE	可選信箱，也可作為匯入者識別。
password_hash	VARCHAR	預留登入功能。
role	VARCHAR	權限角色，預設為 user。
created_at	DATETIME	建立時間。

6.2 players

players 保存棋手資料，避免同一棋手在多盤棋中重複儲存。

欄位	型別	用途
player_id	INT PK	棋手主鍵。
player_name	VARCHAR	棋手名稱。
rank_name	VARCHAR	段位或稱號。
created_at	DATETIME	建立時間。

6.3 game_records

game_records 是對局主表，保存一盤棋的 metadata。

欄位	型別	用途
game_id	INT PK	對局主鍵。
uploader_id	INT FK	匯入者，連到 users。
black_player_id	INT FK	先手棋手，連到 players。
white_player_id	INT FK	後手棋手，連到 players。
event_name	VARCHAR	賽事名稱。
site	VARCHAR	對局地點。
opening	VARCHAR	戰型或開局名稱。
result	VARCHAR	結果，例如 TORYO、DRAW、SENNICHITE。
source_format	VARCHAR	來源格式，目前主要為 CSA。
original_file_name	VARCHAR	原始檔名，用於追溯與去重。
played_at	DATE	對局日期。

6.4 moves

moves 保存每一步棋。

欄位	型別	用途
move_id	INT PK	走法主鍵。
game_id	INT FK	所屬對局。
move_number	INT	第幾手，從 1 開始。
side_to_move	ENUM	下此手的一方，black 或 white。
original_move	VARCHAR	原始走法字串。

欄位	型別	用途
usi_move	VARCHAR	標準化 USI 走法。
move_time	VARCHAR	該手耗時。
comment	TEXT	棋譜註解。

6.5 positions

positions 保存局面。第 0 手是初始局面，第 n 手代表走完第 n 手後的盤面。

欄位	型別	用途
position_id	INT PK	局面主鍵。
game_id	INT FK	所屬對局。
move_number	INT	此局面對應走完第幾手。
side_to_move	ENUM	輪到哪一方行棋。
sfen	TEXT	局面的 SFEN 表示。
sfen_hash	CHAR(64)	SFEN 的 SHA-256 hash。
is_check	BOOLEAN	是否正在被王手。
legal_moves_count	INT	此局面合法手數量。

7 CSA 匯入 MySQL 流程

匯入程式為 `src/import_csa_to_db.py`。它負責把一個 CSA 檔案或整個資料夾中的 CSA 檔案匯入 MySQL。

1. 掃描輸入路徑，支援單檔、資料夾與遞迴掃描。
2. 使用 `cshogi.CSA.Parser` 解析 CSA。
3. 解析棋手名稱、段位、賽事、地點、日期、戰型與結果。
4. 建立或取得 `users` 與 `players` 資料。
5. 建立 `game_records` 對局。
6. 用初始 SFEN 建立 `cshogi.Board`。
7. 對每一步檢查 `board.is_legal(move)`。
8. 寫入 `moves`。
9. 推進棋盤並寫入 `positions`。

10. 每盤棋獨立 commit；錯誤時 rollback，避免半盤棋進入資料庫。

匯入程式也支援 `--dry-run`、`--create-tables`、`--skip-existing`、`--max-games`、`--pause-every` 等參數，使大量資料匯入可以分批進行並避免重複。

8 資料品質與去重

8.1 合法手驗證

資料品質最重要的步驟是合法性檢查。CSA 走法在寫入資料庫前會轉成 cshogi move，並以目前棋盤狀態檢查是否為合法手。若發現非法走法，該盤棋會 rollback，不會留下不完整資料。

8.2 檔名去重

`original_file_name` 可用於快速判斷同名 CSA 是否已經匯入。使用 `--skip-existing` 時，匯入程式會跳過已存在檔名。

8.3 內容去重

只靠檔名不夠，因為同一盤棋可能被重新命名。系統另外使用內容 key：

$$SHA256(\text{initial SFEN} + \text{full USI move sequence})$$

只要初始局面與完整走法序列相同，就視為同一盤棋。這能避免同一棋譜因檔名不同而重複匯入。

9 自動匯入與監控

`src/watch_csa_import.py` 負責監看遠端 live crawler 目錄。遠端爬蟲持續將 CSA 寫入 `$HOME/climbug/data`，watcher 會定期檢查新檔案，等待檔案穩定後複製到 `staging` 目錄，再呼叫 `import_csa_to_db.py` 分批匯入 MySQL。

這樣做有三個優點：

1. 避免爬蟲正在寫檔時就匯入不完整檔案。
2. 透過 batch size 控制每次匯入量，降低資料庫壓力。
3. 用 state file 記錄已處理與失敗檔案，避免重複工作。

10 後端 API 設計

後端位於 `src/csa_browser_api.py`，使用 Python 標準函式庫的 `ThreadingHTTPServer` 與 `BaseHTTPRequestHandler`。它同時支援 MySQL 資料來源與本機 CSA 檔案資料來源。

API	功能
GET <code>/api/games</code>	查詢對局列表，支援事件、日期、棋手、開局等篩選。
GET <code>/api/games/<id>?ply=n</code>	取得某盤棋第 n 手局面，用於棋盤回放。
GET <code>/api/db/stats</code>	回傳棋局、棋手、走法、局面與重複資料統計。
POST <code>/api/upload-csa</code>	接收前端上傳的 CSA，保存到 <code>data/uploads</code> 並匯入 MySQL。
POST <code>/api/self-play/state</code>	根據 USI 手順重播自我對弈狀態。
POST <code>/api/self-play/csa</code>	將自我對弈手順輸出成 CSA 文字。
POST <code>/api/ai-play/state</code>	回傳 AI 對弈目前盤面與 policy 候選手。
POST <code>/api/ai-play/move</code>	讓 AI 搜尋並下一手。
POST <code>/api/policy/candidates</code>	回傳目前局面的 policy 候選手。
POST <code>/api/model-match/step</code>	執行模型互打一個 step。
POST <code>/api/model-match/run</code>	執行多局模型互打。

API 不直接把資料庫 row 原樣送到前端，而是序列化成為適合 UI 使用的 JSON，例如棋盤格、持駒、最後一步、合法手、是否王手、PV 與 policy 候選手。

11 前端棋譜瀏覽器

前端位於 `web/index.html`、`web/app.js` 與 `web/styles.css`。它是一個原生 JavaScript 應用，透過 Fetch API 呼叫後端。

主要功能包含：

- **棋譜列表**：從 `/api/games` 載入對局，支援條件搜尋。
- **棋盤回放**：使用 `/api/games/<id>?ply=n` 跳到任意手數。
- **資料庫統計**：顯示目前 MySQL 中的資料量與重複群組。
- **CSA 上傳**：前端用 multipart form 上傳，後端匯入 MySQL。
- **自我對弈**：使用者可手動下棋並匯出 CSA。

- **AI 對弈**：使用者與 AI 對弈，畫面顯示搜尋評分、深度、節點、PV 與 policy 候選手。
- **模型互打**：選擇兩個 policy model，以相同搜尋框架互相對弈比較。

12 資料庫如何支援 AI

資料庫對 AI 有三個用途。

12.1 訓練資料匯出

src/export_policy_dataset.py 從 MySQL 取出每個 Position_t 與下一手 Move_t+1：

```
SELECT p.game_id, p.move_number, p.sfen, m.usi_move, g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id
  AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;
```

匯出後會產生 states、moves、game_ids、meta 等 NumPy arrays，供 policy network 訓練。

12.2 Opening Book

OpeningBook.from_mysql 從 MySQL 統計前若干手中，同一 SFEN 的常見下一手。若某手出現次數達到門檻，AI 可在開局階段直接採用資料庫中的常見實戰走法。這能降低開局搜尋成本，也讓 AI 的開局更接近人類實戰資料。

12.3 AI 對弈資料服務

AI 對弈時，資料庫提供棋譜回放與訓練來源；模型提供 policy 候選手排序；傳統搜尋負責局面計算。新版系統已移除神經網路 value head，資料庫主要支援 policy 訓練與 opening book。

13 安全性與維護

13.1 SQL 注入防護

Python 查詢使用 PyMySQL 參數化查詢，例如：

```
cursor.execute(
    "SELECT game_id FROM game_records WHERE original_file_name = %s",
    (file_name,)
)
```

這可以避免把使用者輸入直接拼接成 SQL。

13.2 密碼管理

MySQL 密碼不應寫死在程式碼與報告中。程式支援從命令列參數、環境變數或互動輸入取得密碼。啟動瀏覽器批次檔也會在 MySQL 模式詢問密碼。

13.3 上傳安全

CSA 上傳功能限制副檔名為 .csa，限制檔案大小，並以 UUID 產生安全檔名。後端只把檔案寫入 data/uploads，再交給既有匯入流程處理。

14 效能與索引建議

系統常見查詢包含依 game_id 與 move_number 查局面、依檔名去重、依日期或棋手搜尋對局、依 sfen_hash 統計重複局面。建議索引如下：

```
CREATE INDEX idx_game_records_original_file_name
ON game_records(original_file_name);

CREATE INDEX idx_game_records_played_at
ON game_records(played_at);

CREATE INDEX idx_moves_game_number
ON moves(game_id, move_number);

CREATE INDEX idx_positions_game_number
ON positions(game_id, move_number);

CREATE INDEX idx_positions_sfen_hash
ON positions(sfen_hash);
```

idx_positions_game_number 對前端回放最重要，因為每次跳手數都會用到。idx_moves_game_n 對資料集匯出最重要，因為匯出時需要把 position 與下一手 move 對齊。

15 限制與未來改進

目前系統仍有一些限制：

- 匯出訓練資料時中途沒有進度列，資料量大時使用者只能看程序是否還在。
- MySQL 匯入與訓練匯出可能同時發生，需要更完整的排程管理。
- 前端統計仍偏基本，可加入戰型分布、棋手分布、年份分布等分析。
- 資料庫 schema 可加入更多唯一約束與索引，以提高去重與查詢效能。
- 上傳功能目前適合內部展示，若要公開使用仍需登入、權限與檔案掃描。

16 結論

shogiAI 的資料庫系統將原本分散的 CSA 棋譜轉換成可查詢、可回放、可訓練的關聯式資料。遠端爬蟲負責持續蒐集棋譜，watcher 負責分批匯入 MySQL，後端 API 負責把資料轉成前端與 AI 可用的 JSON，前端則提供使用者互動介面。資料庫不只是儲存層，而是串接資料蒐集、棋譜管理、AI 訓練與 AI 對弈的核心基礎。

A 常用操作指令

A.1 匯入 CSA 到 MySQL

```
python src\import_csa_to_db.py ^
--input data ^
--recursive ^
--create-tables ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213 ^
--skip-existing
```

A.2 啟動瀏覽器 API

```
python src\csa_browser_api.py ^
--source mysql ^
--db-host 140.135.65.53 ^
--db-port 3306 ^
```

```
--db-user 11211213 ^  
--db-name DB11211213 ^  
--policy-model models\policy_model.pt ^  
--policy-order-ply 2 ^  
--opening-book-ply 30 ^  
--opening-book-min-count 2
```

A.3 匯出 policy 訓練資料

```
python src\export_policy_dataset.py ^  
--output out\policy_dataset.npz ^  
--host 140.135.65.53 ^  
--port 3306 ^  
--user 11211213 ^  
--database DB11211213
```